

## Problem description

Consider the 1D linear diffusion equation:

$$\frac{\partial u}{\partial t} = \frac{\partial^2 u}{\partial x^2} + \cos(x), \quad x \in [0, \pi], \quad t \in [0, 1] \quad (1)$$

The initial and boundary conditions are:

$$u(x, 0) = 0 \quad (2)$$

$$u(0, t) = 1 - \exp(-t) \quad (3)$$

$$u(\pi, t) = -1 + \exp(-t) \quad (4)$$

Solve the PDE using a PINN and ensure it purely learns from physical constraints.

## 1 Theory

### 1.1 Physics-Informed Neural Networks

#### Role of neural network

A neural network in general works as a universal function approximator and in this case it is trying to predict the sought solution  $u(x, t)$ . Instead of applying traditional and equation based numerical methods like Finite Difference Method (FDM) or Finite Element Method (FEM) the neural network is trained to approximate the true solution function.

$$u_\theta(x, t) \approx u(x, t)$$

Here  $\theta$  denotes the trainable parameters e.g. the weights and biases of the network. The network takes the spatial and temporal domain as an input and maps them onto a predicted solution as an output.

#### Incorporation of PDE and BC into loss function

In order to force the neural network to comply with the governing equations, mostly PDE's, the training objective of minimizing the loss is altered. In data based methods the neural network is trained to fit a function that matches the training data and the loss function is constructed accordingly. In the case of PINN the loss function is separated into different parts, namely the PDE residual term, the boundary condition term and in cases of initial value problems also the initial condition term. The PDE residual contributes to the loss function in the way that it is a measurement of how well the predicted solution complies with the PDE. The contribution of the initial condition and boundary condition loss is a measurement of how well the predicted solution aligns with the initial and boundary condition. The goal of the neural network is to optimize the parameters  $\theta$  iteratively and minimize the loss value accordingly.

#### Importance of collocation points

Collocation points play a crucial role in Physics-Informed Neural Networks (PINNs) because they serve as the locations where the PDE residual is enforced. The collocation points represent the discretization of the domain, but they feature an important advantage. Because they can be randomly sampled over the domain and can adaptively sampled according to the characteristics of the problem. Another advantage of this discretization problem over traditional discretization methods (FDM, FEM) is the lower computational cost in complex high-dimensional problems, where they do not suffer from the "curse of dimensionality" as much.

## 1.2 Advantages and drawbacks of PINN approach

### Advantages

As mentioned with the importance of collocation points, the vanished necessity of a mesh grid is a advantage due to the easy adaptability also on complex geometries. This discretization method in cooperation with the neural network predicts a continuous solution instead of a discrete solution. This is reasoned in the automatic differentiation, that computes exact derivatives with respect to the inputs, which leads to higher accuracy for smooth solutions.

Another mentioned advantage is the better scalability to high-dimensional PDE's, where traditional numerical methods struggle to cope with the exponentially growing computational cost.

A striking advantage in research applications is the easy incorporation of experimental data, which can help the neural network to be guided towards feasible solutions in complex problems (hybrid approach). A different possible advantage of incorporating observed or experimental data is the application of inverse problems.

### Drawbacks

In case of large-scale problems the training of the neural network requires much more computational cost in comparison to traditional numerical methods. The difference in computation time is very large for linear PDE's, where traditional numerical methods are able to find a solution in a single step and neural networks require thousands of iterations to converge.

The optimization objective often combines multiple constraints, which can effect the training dynamics. It is possible that some parts of the solution are learned faster than others.

While the prediction of a continuous solution is an advantage for smooth solutions, it is a drawback for problems featuring discontinuities or sharp gradients. In these cases the neural network takes much longer to converge and often requires adaptive sampling or neural operators to improve its performance.

In traditional numerical methods the performance of solving a PDE based problem is highly dependent on the method itself and how it respects the characteristics of the featured problem. In the case of PINN this challenge is analogous to the hyperparameter sensitivity. Where in traditional numerical methods it is possible to draw a connection between the best possible approximation scheme and the PDE and it is extensively researched, for the PINN the outcome of the hyperparameter search is much less predictable. This problem is connected to the lack of explainability of neural networks that often have to be treated as a blackbox.

As a final drawback of the PINN approach, the lack of guaranteed convergence is crucial for the design choice for the architecture of the neural network. While in theory a finite number of neurons from a single layer are able to solve a well-posed problem, in practice this is most of the time not applicable and not the most computational cost efficient way to deal with PDE's. The neural network is in best case scenario designed in a way that its possible to cope with the complexity of the problem, but not more in order to prevent over-fitting. Over fitting is only a problem that would arise in inverse or hybrid approaches, as this is a consequence of data dependency.

## 2 Implementation

### 2.1 Neural network architecture

The design of the neural network architecture is crucial for the production of accurate and cost efficient results. The architecture does not only depend on the input shape, the output shape, the number of hidden layer and the number of neurons per layer, but also the activation function and the learning rate. In order to investigate the influence of the hyperparameter, a hyperparameter analysis is conducted. In this analysis the resulting loss values are evaluated against each other, where convergence rate as well as stability are considered important factors. The analysis is conducted featuring the following design choices that stayed the same throughout the hyperparameter analysis:

|                                    |       |
|------------------------------------|-------|
| Input shape                        | 2     |
| Output shape                       | 1     |
| No. of inner collocation points    | 1000  |
| No. of initial condition coll. p.  | 200   |
| No. of boundary condition coll. p. | 200   |
| No. of epochs                      | 10000 |
| No. of neurons per layer           | 32    |
| Optimizer                          | Adam  |

To make a feasible architecture design choices the following hyperparameters are iteratively changed in separate trainings, so that every possible combination is used for solving the problem:

|                      |                       |
|----------------------|-----------------------|
| Learning rate $\eta$ | [0.01, 0.001, 0.0001] |
| No. of layer         | [2, 3, 4, 5]          |
| Activation function  | [tanh, ReLu]          |

One thing that is evident immediately from the hyperparamter analysis, is that the tanh activation function is working significantly better than the ReLu. Furthermore the effect of the learning rate was pretty significant. While a lower learning rate leads to small loss values in a smaller amount of epochs, the learning progress is very instable and high loss values eventually arise at a high number of epochs. With all the candidate numbers of neurons per layer the neural network is able to cope with the complexity of the problem. That being sad, three layer performed significantly better than two. Therefore the following design choices are made:

|                      |        |
|----------------------|--------|
| Learning rate $\eta$ | 0.0001 |
| No. of layer         | 3      |
| Activation function  | tanh   |

All loss evolutions for the 12 conducted PINN runs can be seen in the appendix. Since the stability of the small learning rate is desired, the design choice is accompanied by a doubling of the epochs in order to achieve convergence. The loss evolution of the chosen architecture thta features 2240 trainable parameters is presented in figure 1.

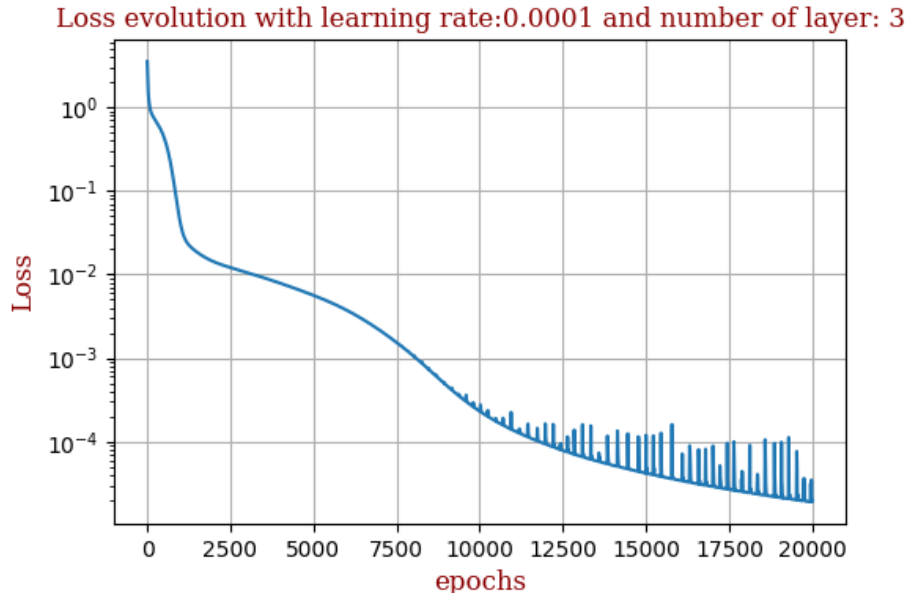


Figure 1

## 2.2 Loss function for 1D heat equation

As mentioned in the section about the incorporation of the PDE and boundary condition into the loss function, the loss function for this particular PINN is comprised by three loss components. The PDE residual term, the boundary condition term and the initial condition term.

The PDE residual term reads according to the 1D linear diffusion equation (eq.1) as follows:

$$R(x, t) = u_t - u_{xx} - \cos(x)$$

The residual is contributing to the loss function by its mean squared error (MSE) (note that n is defined by the number of collocation points):

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2, \quad \mathcal{L}_{PDE} = \text{MSE}(R(x, t)) \quad (5)$$

The residual terms are all designed in the way that the deviations from 0 are equivalent to the error, and since the MSE applies to all residuals, the loss contribution from the residual terms is the MSE with  $Y_i = 0$ .

The boundary condition loss according to equation 3 and 4 reads:

$$\mathcal{L}_{BC} = \text{MSE}(u_{pred}(0,t) - u(0,t)) + \text{MSE}(u_{pred}(\pi,t) - u(\pi,t))$$

and the initial condition loss according to equation 2:

$$\mathcal{L}_{IC} = \text{MSE}(u_{pred}(x,0) - u(x,0))$$

The total loss function is a summation over all loss contributions:

$$\mathcal{L} = \mathcal{L}_{PDE} + \mathcal{L}_{BC} + \mathcal{L}_{IC}$$

## 2.3 Training loop

The training loop in a PINN is crucial in the way that it is responsible for iteratively updating the neural network's parameters so that the predicted solution  $u_\theta(x,t)$  satisfies the given PDE, initial conditions, and boundary conditions. To satisfy this over the whole given domain of time and space collocation points are used to calculate the loss, which then is used to perform an optimization step.

### Sampling of collocation points

PINNs do not rely on structured grids, instead, they sample a set of collocation points where the PDE residual is evaluated and enforced to decrease. The points are divided into three categories:

1. Interior points
  - Randomly sampled inside the domain  $(x,t) \in [0,\pi] \times [0,1]$
  - Used to evaluate and enforce the PDE residual
2. Initial condition points
  - Sampled at  $t = 0$  with random  $x \in [0,\pi]$
  - Used to evaluate and enforce the initial condition
3. Boundary condition points
  - at  $x = 0$ , random values in  $t \in [0,1]$ , evaluate and enforce  $u(0,t) = 1 - \exp(-t)$
  - at  $x = \pi$ , random values in  $t \in [0,1]$ , evaluate and enforce  $u(\pi,t) = -1 + \exp(-t)$

All types of collocation points are sampled using uniform random sampling.

### Optimization

After the loss is calculated, the gradients of the loss with respect to the networks weights and biases are computed via backpropagation. This gradient is then used to optimize the parameters weights and biases. This is done using the adaptive moment estimation (Adam) optimizer. The parameter update follows the following definitions and equations, where  $m_\theta$  is the weighted first moment estimate ( $\hat{m}_\theta$  corrected),  $v_\theta$  is the weighted second raw moment estimate ( $\hat{v}_\theta$  corrected),  $\beta_1, \beta_2$  are exponential decay rates for the moment estimates,  $\epsilon$  is a small number to prevent division by zero and  $\theta$  is the updated parameter.

$$\begin{aligned} m_\theta^{(k+1)} &:= \beta_1 m_\theta^{(k)} + (1 - \beta_1) \nabla_\theta \mathcal{L}^{(k)} \\ v_\theta^{(k+1)} &:= \beta_2 v_\theta^{(k)} + (1 - \beta_2) (\nabla_\theta \mathcal{L}^{(k)})^2 \\ \hat{m}_\theta &= \frac{m_\theta^{(k+1)}}{1 - \beta_1^k} \\ \hat{v}_\theta &= \frac{v_\theta^{(k+1)}}{1 - \beta_2^k} \\ \theta^{(k+1)} &:= \theta^{(k)} - \eta \frac{\hat{m}_\theta}{\sqrt{\hat{v}_\theta} + \epsilon} \end{aligned}$$

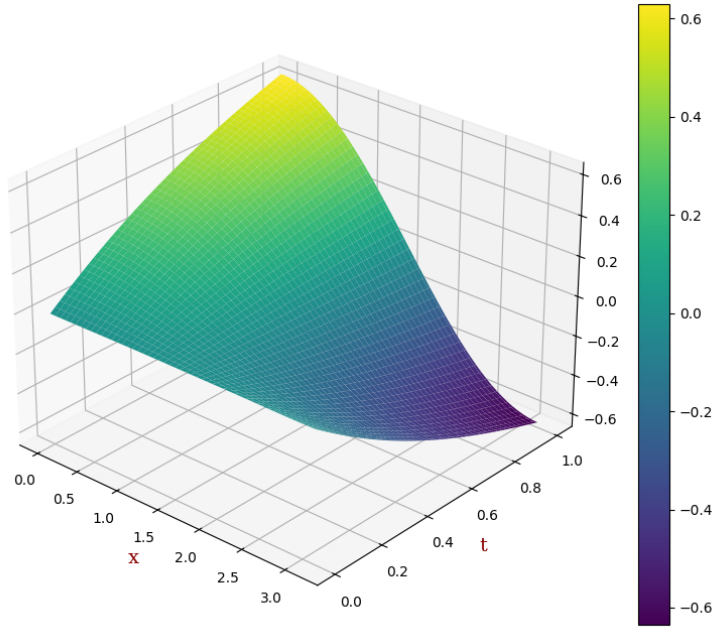
## Evaluation after training

As seen in the reasoning of the chosen neural network architecture, the evolution of the loss value is one measure to evaluate the successive training of a neural network. It is important to ensure convergence on this loss value as this is not guaranteed and shows that the neural network in the chosen hyperparameter setup is capable to approximate the sought solution.

## 3 Validation and Analysis

As a successive minimization of an optimization objective does not necessarily equal to correct predicted solution, it is important to also validate the obtained results. If it is available, it is optimal to validate the predicted solution against an exact solution. In the case of the considered 1D inhomogeneous linear diffusion problem the exact solution is obtainable and can be seen in figure 2 and reads:

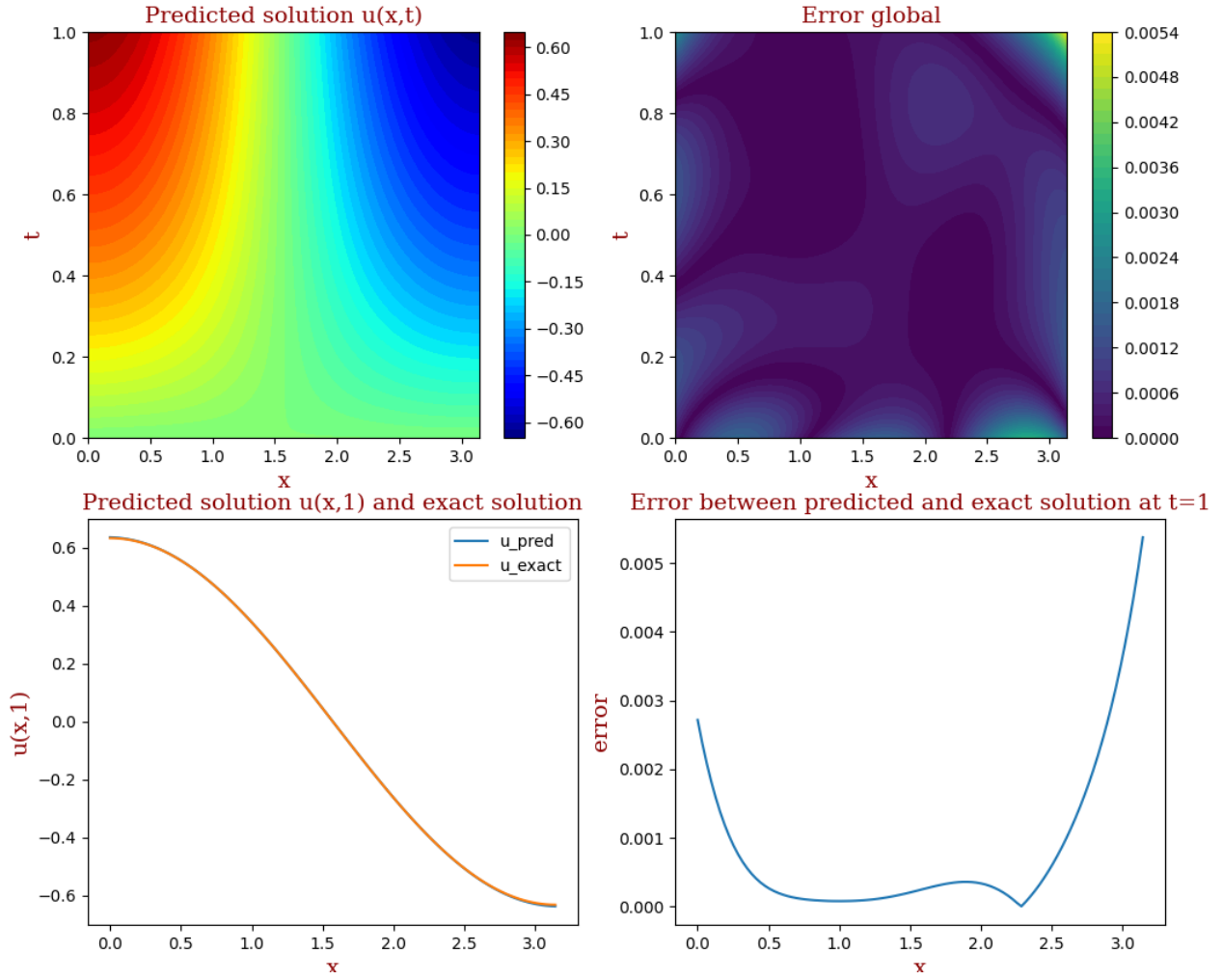
$$u_{exact}(x,t) = \cos(x) - \cos(x) \exp(-t) \quad (6)$$



**Figure 2:** Exact solution plot

### 3.1 Validation

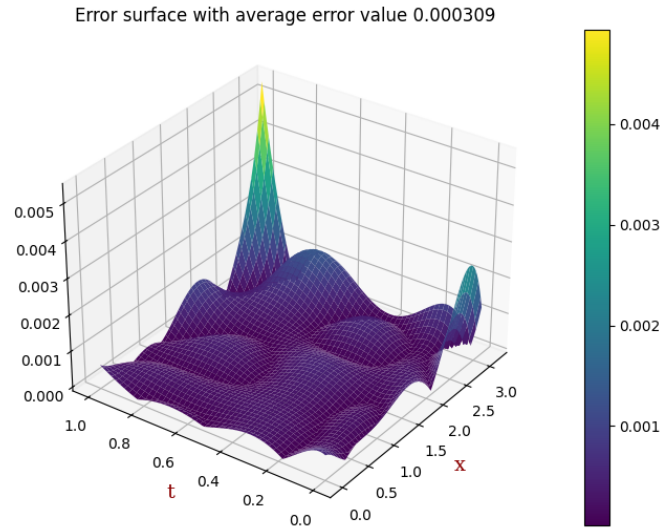
In order to have a visual intuition for the predicted and exact solution and to resulting error, some plots are produced. First of all a contour plot presenting the solution over temporal and spatial domain, where the color indicates the solution value at this point. Then a contour plot showing the error distribution over the domain, where the error measures the difference between the predicted and exact solution. Similarly two one dimensional plots are produced focusing on the solution at the last point in time. As it can be seen from the contour error plot and the one dimensional error plot the error at the boundary of the last time point is the highest.



Generally the error between the exact solution and the predicted solution can be evaluated using different quantitative metrics. One is to calculate the average error, another one is to just focus on the maximum error occurring in the solution.

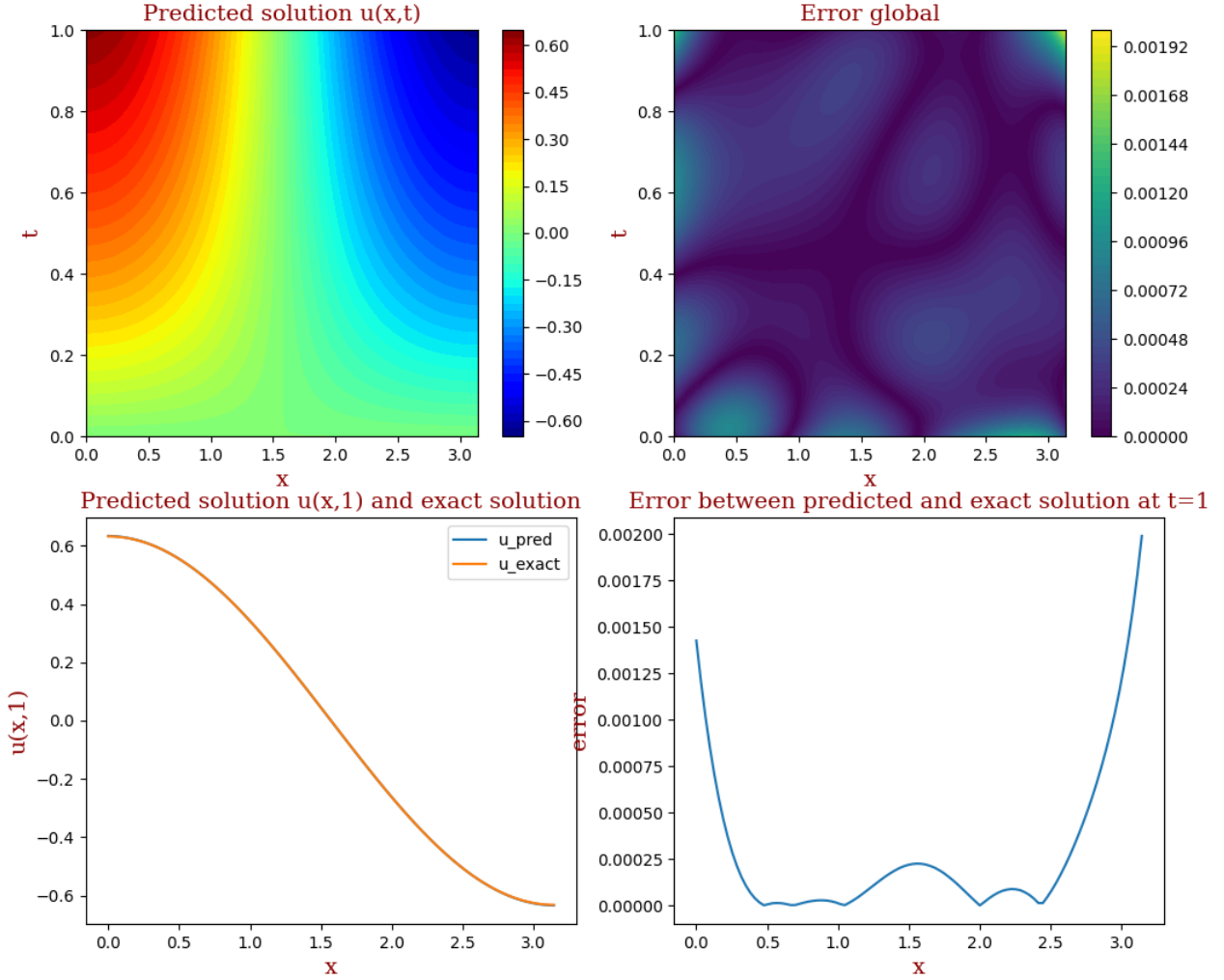
$$error_{avg} = \frac{1}{N} \sum_{i=1}^N error_i, \quad error_{max} = ||error_i||_{max}$$

In the following three dimensional error plot are the maximum occurring error as well as the average error visible:



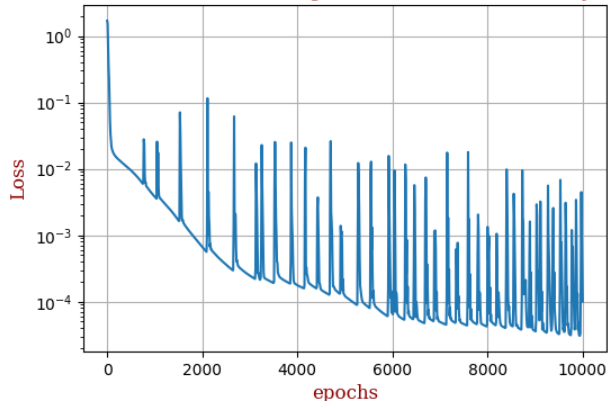
### 3.2 Improvements

As it is already mentioned the highest error values occur at the boundary of the space domain and at the last point of the time domain. And it is also already mentioned that a minimized loss function consisting of multiple loss contributions can result in different accuracies regarding the PDE term, the initial condition and the boundary condition. This is likely to be the case here as well. To cope with this phenomenon it is either possible to sample collocation points at critical temporal and spatial areas more dense, or to give the different losses weights to enforce them. To try out the last attempt of mitigating this effect, one training run is conducted featuring a boundary condition loss weighting with a factor of five. The results show that, while this effect does not vanish, the absolute maximum error decreased.

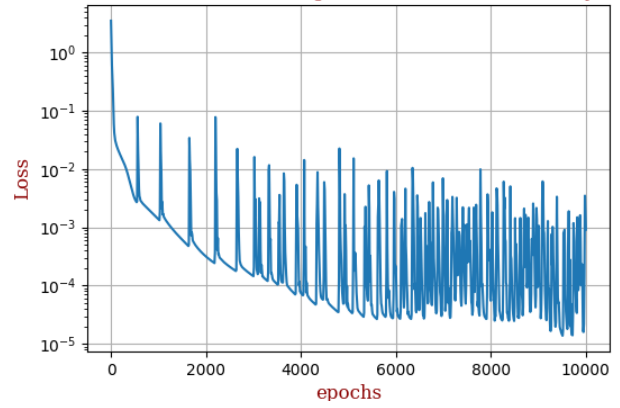


## Appendix

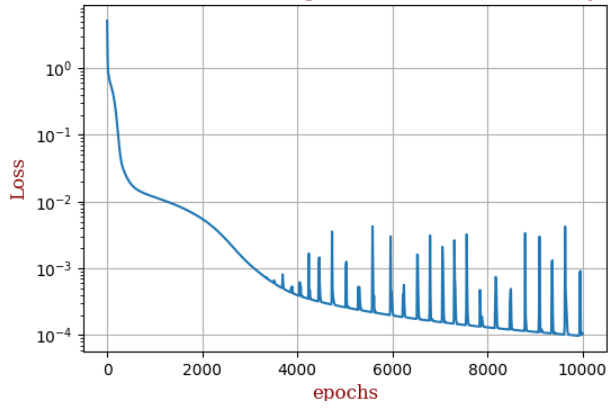
Loss evolution with learning rate:0.01 and number of layer: 2



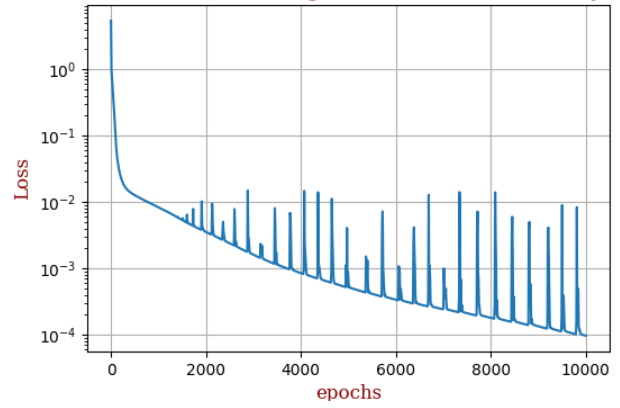
Loss evolution with learning rate:0.01 and number of layer: 3



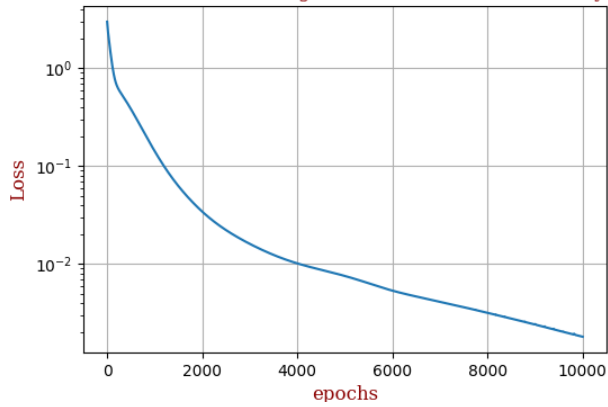
Loss evolution with learning rate:0.001 and number of layer: 2



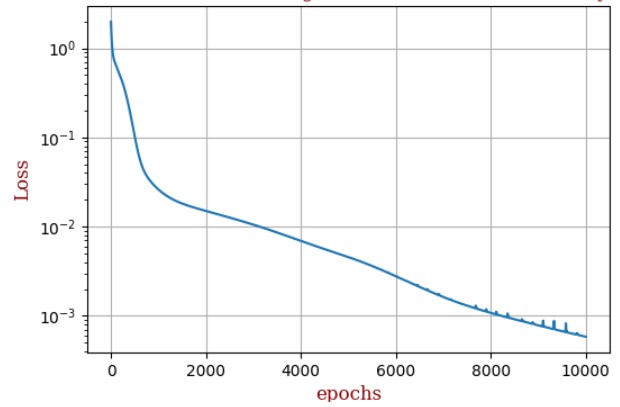
Loss evolution with learning rate:0.001 and number of layer: 3



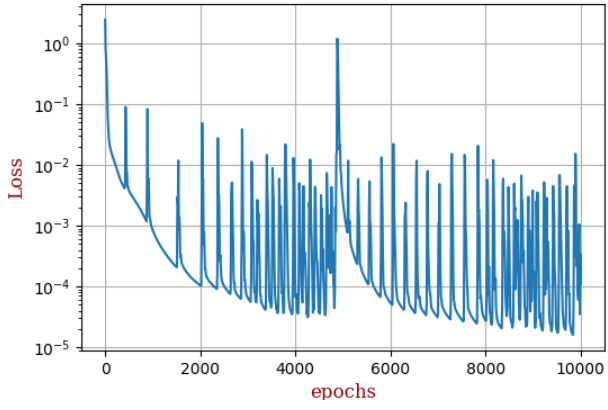
Loss evolution with learning rate:0.0001 and number of layer: 2



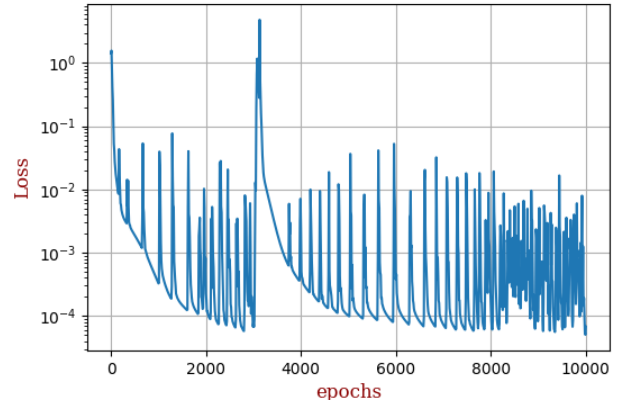
Loss evolution with learning rate:0.0001 and number of layer: 3



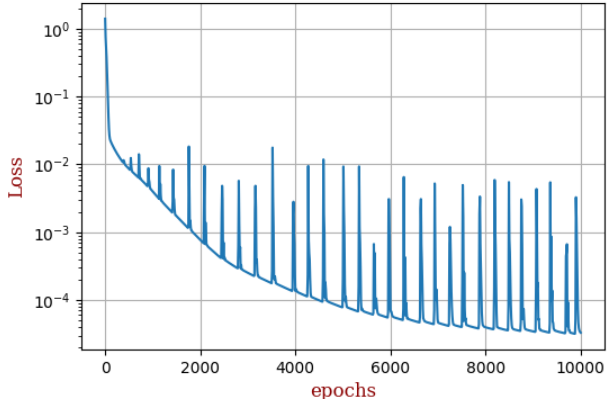
Loss evolution with learning rate:0.01 and number of layer: 4



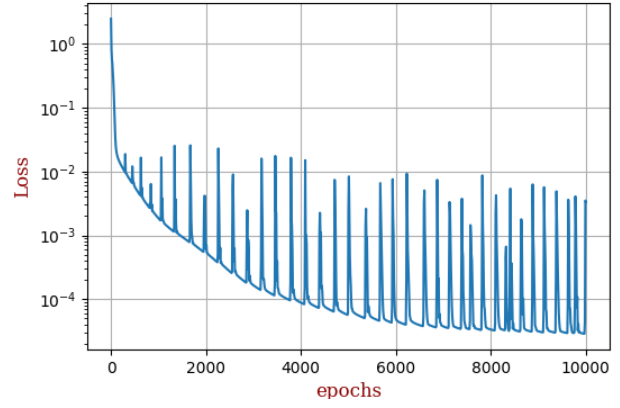
Loss evolution with learning rate:0.01 and number of layer: 5



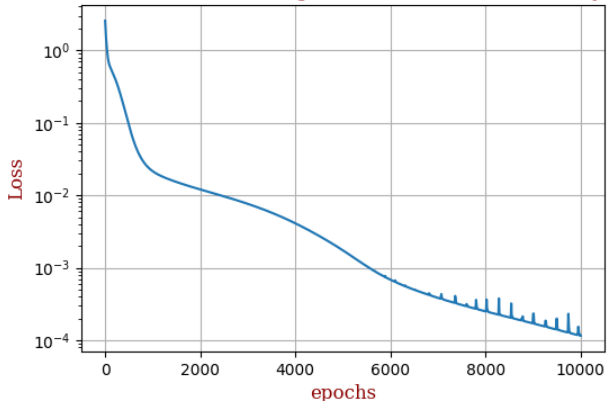
Loss evolution with learning rate:0.001 and number of layer: 4



Loss evolution with learning rate:0.001 and number of layer: 5



Loss evolution with learning rate:0.0001 and number of layer: 4



Loss evolution with learning rate:0.0001 and number of layer: 5

